1) Right-Angled Triangle Star Pattern

Problem Statement

A school wants to display a **Right-Angled Triangle pattern** using stars (*) for an annual event decoration.

Write a Python program that prints a **right-angled triangle pattern** of stars for a given integer n.

The triangle should:

- Start with **1 star** in the first row.
- Increase by **1 star in each row**.
- End with n stars in the last row.

Example:

For n = 5

```None
*
**
***
****
*****
```

Input Format

A single integer n representing the number of rows.

Example:

```None
5
```

Output Format

Print the **Right-Angled Triangle Star Pattern**.

Constraints

```None
1 ≤ n ≤ 50
```

Test Case 1

Input:

```None
5
```

Output:

```None
*
**
***
****
*****
```

Explanation

For `n = 5`:

- Row 1 → print `1` star
- Row 2 → print `2` stars
- Row 3 → print `3` stars
- Continue until row `n`

Test Case 2

Input:

```None
3
```

Output:

```None
*
**
***
```

Test Case 3

Input:

```None
1
```

Output:

```None
*
```

Test Case 4

Input:

```None
7
```

Output:

```None
*
**
***
****
*****
******
*******
```

Test Case 5

Input:

```None
6
```

Output:

```None
*
```

```
**
***
****
*****
******
```

Test Case 6

Input:

```None
4
```

Output:

```None
*
**
***
****
```

Test Case 7

Input:

8

Output:

*

```
**
***
****
*****
******
*******
********
```

Python Solution

```python
n = int(input())

for i in range(1, n + 1):
    print("*"*i)
```

2) Largest Among Two Numbers Using If Statement

Problem Statement

A company is developing a small comparison system to determine which of two employee performance scores is higher.

Write a Python program that takes **two integers** as input and prints the **largest number** using **only the `if` statement**.

Rules:

- If the first number is greater than the second number, print the first number.

- If the second number is greater than the first number, print the second number.
- If both numbers are equal, print `"Both are Equal"`.

Example:

For input:

```None
10 20
```

Output:

```None
Largest Number: 20
```

Input Format

A single line containing **two integers separated by space**.

Example:

```None
10 20
```

Output Format

Print exactly:

```None
Largest Number: <number>
```

OR

```None
Both are Equal
```

Constraints

```None
-10^6 ≤ number ≤ 10^6
```

Test Case 1

Input:

```None
10 20
```

Output:

```None
Largest Number: 20
```

Explanation

For input `10 20`:

- `20` is greater than `10`
- So the output is:

```None
Largest Number: 20
```

Test Case 2

Input:

```None
50 25
```

Output:

```None
Largest Number: 50
```

Test Case 3

Input:

```None
100 100
```

Output:

```None
Both are Equal
```

Test Case 4

Input:

```None
-5 -10
```

Output:

```None
Largest Number: -5
```

Test Case 5

Input:

```None
0 10
```

Output:

```None
Largest Number: 10
```

Test Case 6

Input:

```None
-20 15
```

Output:

```None
Largest Number: 15
```

Test Case 7

Input:

```None
999 1000
```

Output:

```None
Largest Number: 1000
```

Python Solution

```Python
a, b = map(int, input().split())

if a > b:
    print("Largest Number:", a)

if b > a:
    print("Largest Number:", b)

if a == b:
    print("Both are Equal")
```

3) Sum of First N Natural Numbers Using For Loop

Problem Statement

A mathematics teacher wants to calculate the **sum of the first `N` natural numbers** for classroom practice.

Write a Python program that takes an integer `N` as input and calculates the **sum from 1 to N** using a **for loop**.

Rules:

- Use a **for loop only**.
- `N` will always be greater than `0`.
- Print the final sum in the required format.

Example:

For `N = 5`

```None
1 + 2 + 3 + 4 + 5 = 15
```

Input Format

A single integer `N`.

Example:

```None
5
```

Output Format

Print exactly:

```None
Sum: <sum>
```

Constraints

```None
1 ≤ N ≤ 10^6
```

Test Case 1

Input:

```None
5
```

Output:

```None
Sum: 15
```

Explanation

For `N = 5`:

- Add all numbers from `1` to `5`
- `1 + 2 + 3 + 4 + 5 = 15`

So output is:

```None
Sum: 15
```

Test Case 2

Input:

```None
10
```

Output:

```None
Sum: 55
```

Test Case 3

Input:

```None
1
```

Output:

```None
Sum: 1
```

Test Case 4

Input:

```None
7
```

Output:

```None
Sum: 28
```

Test Case 5

Input:

```None
20
```

Output:

```None
Sum: 210
```

Test Case 6

Input:

```None
2
```

Output:

```None
Sum: 3
```

Test Case 7

Input:

```None
15
```

Output:

```None
Sum: 120
```

Python Solution

```Python
n = int(input())

total = 0

for i in range(1, n + 1):
    total += i

print("Sum:", total)
```

4) Count Even and Odd Numbers Using Conditional and Control Flow Statements

Problem Statement

A school wants to analyze student roll numbers for an event.

Write a Python program that takes an integer `N` and counts how many numbers from `1 to N` are **Even** and **Odd** using:

- **For Loop** (Control Flow Statement)
- **If Condition** (Conditional Statement)

Rules:

- Traverse numbers from `1 to N`
- If a number is divisible by `2`, count it as **Even**
- Otherwise, count it as **Odd**
- Print the final counts in the required format

Example:

For `N = 5`

Numbers:
`1 2 3 4 5`

Even Numbers → `2, 4` → Count = `2`
Odd Numbers → `1, 3, 5` → Count = `3`

Input Format

A single integer `N`.

Example:

```None
5
```

Output Format

Print exactly:

```None
Even Count: <count>
Odd Count: <count>
```

Constraints

```None
1 ≤ N ≤ 10^6
```

Test Case 1

Input:

```None
5
```

Output:

```None
Even Count: 2
Odd Count: 3
```

Explanation

For `N = 5`:

- 1 → Odd
- 2 → Even
- 3 → Odd
- 4 → Even
- 5 → Odd

Final Count:

- Even = 2
- Odd = 3

Test Case 2

Input:

```None
10
```

Output:

```None
Even Count: 5
Odd Count: 5
```

Test Case 3

Input:

```None
1
```

Output:

```None
Even Count: 0
Odd Count: 1
```

Test Case 4

Input:

```None
7
```

Output:

```None
Even Count: 3
Odd Count: 4
```

Test Case 5

Input:

```None
20
```

Output:

```None
Even Count: 10
Odd Count: 10
```

Test Case 6

Input:

```None
2
```

Output:

```None
Even Count: 1
Odd Count: 1
```

Test Case 7

Input:

```None
15
```

Output:

```None
Even Count: 7
Odd Count: 8
```

Python Solution

```python
n = int(input())

even_count = 0
odd_count = 0

for i in range(1, n + 1):

    if i % 2 == 0:
        even_count += 1
    else:
        odd_count += 1

print("Even Count:", even_count)
print("Odd Count:", odd_count)
```

5) Multiplication Table Generator Using Conditional and Control Flow Statements

Problem Statement

A school mathematics teacher wants to generate a **multiplication table** for students.

Write a Python program that takes an integer N and prints its multiplication table from **1 to 10** using:

- **For Loop** (Control Flow Statement)
- **If Condition** (Conditional Statement)

Rules:

- If N is **greater than 0**, print the multiplication table.

- If N is **less than or equal to 0**, print:

```
None
Invalid Number
```

- The multiplication table should be printed in the format:

```
None
N x i = result
```

Example:

For N = 5

```
None
5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50
```

Input Format

A single integer N.

Example:

```None
5
```

Output Format

If valid:

```None
N x 1 = result

...

N x 10 = result
```

Otherwise:

```None
Invalid Number
```

Constraints

```None

-10^6 ≤ N ≤ 10^6
```

Test Case 1

Input:

```None
5
```

Output:

```None
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Explanation

For `N = 5`:

- Since `5 > 0`, print multiplication table from `1 to 10`.

Test Case 2

Input:

```None
2
```

Output:

```None
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

Test Case 3

Input:

```None
1
```

Output:

```None
1 x 1 = 1

1 x 2 = 2

1 x 3 = 3

1 x 4 = 4

1 x 5 = 5

1 x 6 = 6

1 x 7 = 7

1 x 8 = 8

1 x 9 = 9

1 x 10 = 10
```

Test Case 4

Input:

```None
10
```

Output:

```None
10 x 1 = 10

10 x 2 = 20

10 x 3 = 30

10 x 4 = 40

10 x 5 = 50

10 x 6 = 60

10 x 7 = 70

10 x 8 = 80

10 x 9 = 90

10 x 10 = 100
```

Test Case 5

Input:

```None
0
```

Output:

```None
Invalid Number
```

Test Case 6

Input:

```None
-5
```

Output:

```None
Invalid Number
```

Test Case 7

Input:

```None
7
```

Output:

```None
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
```

```
7 x 6 = 42

7 x 7 = 49

7 x 8 = 56

7 x 9 = 63

7 x 10 = 70
```

Python Solution

Python

```python
n = int(input())


if n > 0:

    for i in range(1, 11):

        print(n, "x", i, "=", n * i)

else:

    print("Invalid Number")
```